

Cognee proposal: a semantic core for portable agent memory

Discussion draft v0.1 | September 9, 2026 | Cognee contribution to OMPI

Status: Proposed for discussion; not an adopted OMP standard. COGX 0.1 is a working implementation. Most guarantees proposed here are not yet implemented.

1. What OMP should standardize

Portability requires more than exchanging text. A receiver needs to know what each object represents, its origin and context, whether it may be used, how it relates to prior material, and what the transfer changed. OMP should standardize that meaning through one common contract:

- **Identity and revision:** stable origin-qualified identities for memory objects and their immutable revisions.
- **Content, origin, and evidence:** an explicit distinction between source material, transcripts, human assertions, and machine-derived memory.
- **Scope, ownership, and authority:** separate descriptions of whom a memory concerns, where it applies, who controls it, and who may act on it.
- **Provenance and lineage:** traceable relationships from evidence through extraction, correction, consolidation, and supersession.
- **Time, lifecycle, and deletion:** shared meanings for validity, invalidation, expiry, removal, and protection against unintended resurrection.
- **Exchange capability and fidelity:** explicit transfer intent, supported semantics, conflicts, identity mappings, and information loss.

Files, graphs, databases, models, retrieval, embeddings, prompts, and internal mutation remain implementation choices. Conformance preserves declared meaning and controls; it does not require identical graphs or answers.

Section 2 relates this contract to the other contributions. Section 3 defines its six parts in the same order; Sections 4–6 evaluate COGX, identify profiles above the core, and propose an interoperability program.

2. Relationship to the existing work

The proposals are strongest at different layers, not as competing definitions of the whole protocol. The [scoping note](#) already identifies the narrow waist: a portable memory record plus consolidation lineage, with storage and retrieval left to implementations. This proposal makes it testable.

Packer / Letta: a Markdown loading profile

The [Packer draft](#) defines a useful coding-agent profile: root Markdown is prompt-visible, nested material is deferred, and the agent can discover and selectively read it.

File depth cannot reliably carry identity, provenance, policy, or lifecycle, and making it universal would exclude server-backed and structured systems for no interoperability gain. OMP should preserve the directory contract as a profile of common objects. Its projections must retain required identifiers and metadata or declare omissions. Imported text does not gain instruction authority merely by becoming prompt-visible; the harness controls authorized loading.

IBM: object and runtime foundations

The [IBM draft](#) contributes source-local identity, human and AI provenance, source references, scope and ACL mapping, temporal invalidation, and immutable history. Its remember, recall, observe, and decorate operations belong in a runtime binding.

Immutability should apply to exchanged revisions and lifecycle events, not internal rows or files. The draft's one-system-at-a-time validity rule is narrower than portability requires: copies and federation need the same origin identity and lineage across systems. Copy, move, and federation must be distinct.

AIDP: a federation binding

The [AIDP draft](#) correctly distributes memory across servers and separates files, transcripts, and inferences. Its transport-independent endpoints and MCP integration are a natural runtime binding.

Endpoints alone cannot preserve meaning or controls. If every operation may be disabled and permissions remain server-local, conformance has no useful floor. The binding needs capabilities, explicit write destinations, common identity and lifecycle, policy failures, and per-server receipts. We use source evidence rather than “ground truth”: provenance establishes origin, not truth.

Complementary controls and prototypes

The [Agent Memory System Card](#) documents behavior and limitations; it does not define interchange. The [me.md proposal](#) contributes a valuable user-authority profile, including write consent, provenance, and revocation ideas, without requiring its governance choices in every deployment.

The [Python loader](#) implements the Packer contract; the [ACP memory-server prototype](#) normalizes and searches coding-agent transcripts. Neither yet demonstrates a shared object, policy, and lifecycle contract across implementations. APIs make memory reachable; shared semantics make that access interoperable.

3. Meaning of the common core

The requirements below define the intended semantics. Exact field names, serialization, minimum vocabularies, and extension mechanisms remain working group decisions.

3.1 Identity and revision

Every object must carry a logical identity qualified by its origin authority and namespace, together with an immutable revision identity. A receiving system may assign local identifiers, but it must retain the origin-to-local mapping. A correction produces a new revision or replacement linked to its predecessor; it does not silently rewrite the exchanged history.

Identity is not text equality. Matching content does not merge distinct revisions; different content under one revision identity is a conflict. Internal storage may remain mutable.

3.2 Content, origin, and evidence

A conforming profile must distinguish source artifacts, transcripts, human-authored assertions, and machine-derived memories. These are epistemic roles, not truth labels: an utterance is evidence of what was said, not proof of its claim. A derived memory with unavailable evidence must say so.

Text should be the baseline representation. Other media may be embedded or referenced through typed descriptors that make their media type and integrity verifiable. Profiles may introduce entities, facts, summaries, memory blocks, or domain objects, provided they preserve the object's origin role when mapping to and from the common core.

3.3 Scope, ownership, and authority

The contract must distinguish a memory's subject, owner, human author or generating agent, origin system, and applicable context. Profiles should support user, agent, session, run, project, team, and organization scopes, with qualified extensions. A scope label is neither an access grant nor proof of ownership.

Portable policy identifies principals, resources, actions, and retention or use constraints. Importers map principals and enforce source constraints with destination policy. Unmapped identities or unsupported required rules cause rejection or restricted staging, never silent access expansion. Credentials (passwords and hashes, API keys, and tokens) are outside memory exchange and require a separate administrative operation. Permission to know is not permission to act; permission to import is not permission to treat content as an instruction.

3.4 Provenance and lineage

A derivation must reference the exact input revisions and, where available, the relevant transcript turns or source spans. It should record the transformation, responsible actor or model, time, and available configuration identifier. Unknown provenance must remain unknown. The model should map to [W3C PROV](#) entities, activities, agents, derivation, and revision instead of inventing an isolated vocabulary. [PROV invalidation](#) ends an entity's availability; it does not express epistemic retraction or a proposition's application interval. OMP must define supersession, retraction, and application-time validity separately.

Consolidation must name all inputs; a summary cannot erase its relationship to them. Re-derivation links to what was actually processed, even when that input was derived. Lineage need not retain deleted content indefinitely, but its surviving relationship and limits must be explicit.

3.5 Time, lifecycle, and deletion

Recording time is distinct from when a claim applies. The contract distinguishes active, superseded, invalidated, expired, and deleted states. Each lifecycle event carries an identity, target revision, authorized actor, effective time, and available reason. Concurrent or unordered events surface as conflicts. Current reads exclude invalidated and expired claims; historical reads are explicit and permission-checked.

Deletion declares coverage over source objects, derivations, indexes, and caches under the receiver's control. Dependents are deleted, invalidated, or recomputed according to policy. A permitted tombstone may prevent resurrection; it and remaining lineage stay policy-bound. Receipts distinguish requests, completed erasure, and unresolved replicas. Offline exchange cannot erase an unreachable system.

For example, if one transcript supports Tuesday status updates and a later one changes the preference to Friday, the second revision supersedes the first from its effective time and a summary links to both. An authorized copy preserves those links and controls: current reads return Friday; historical reads can explain Tuesday. If evidence is deleted, policy governs its derivations, and a permitted tombstone prevents an old export from making them current again.

3.6 Exchange capability and fidelity

Every exchange begins with a manifest declaring format and profile versions, object kinds, scope coverage, required extensions, and complete snapshot versus selection. Absence from a partial export never implies deletion. A receiver must reject or restrict objects whose required policy or lifecycle semantics it cannot enforce; opaque retention is not the same as understanding.

Import behavior is explicit. Preserve retains supported meaning without new inference. Re-derive creates new identities linked to actual inputs. Hybrid distinguishes retained from new objects. These are semantic outcomes, not algorithms, and do not promise identical retrieval or answers.

An object-level receipt identifies accepted, transformed, omitted, unresolved, and rejected objects, with identity mappings and reasons. Missing evidence, broken references, unsupported extensions, policy failures, and partial results must be visible; partial failures state retry boundaries. Retries must not duplicate objects or events.

Transfer intent is also explicit. A copy leaves authorized source objects in place. A move deletes them only after an authorized, durable, conforming destination import; a failed move remains incomplete. Federation provides authorized access where objects reside. Connecting a server implies neither replication nor write access.

4. COGX as a reference implementation

COGX 0.1, the Cognition eXchange format, is a running workbench for these ideas. Its inspectable exchange directory contains `manifest.json` and populated JSONL streams; it may be packed as `.cogx.tar.gz` for transport. This is an artifact layout, not a required storage architecture.

Typed records cover documents, episodes, entities, facts, atomic memories, and named memory blocks, with external identifiers, timestamps, metadata, and user, agent, session, and run scopes. A raw-node record retains unmapped Cognition graph properties as an optional extension, not a universal memory kind. See the [record models](#) and [archive transport](#).

[Adapters](#) translate Mem0, LangMem, Letta, Zep/Graphiti, and COGX imports into these records; they do not establish bidirectional provider compatibility. The [loader](#) implements `preserve`, `re-derive`, and `hybrid`: retain supported graph structure without LLM extraction, extract again from text, or do both. Vector indexing may still require embeddings.

An authorized Cognition-to-Cognition directory transfer is already concrete:

```
import cognition
from cognition.migration import COGXArchiveSource

archive_dir = "./project_memory_cogx"
await cognition.export("project_memory", format="cogx", destination=archive_dir)
await cognition.remember(
    COGXArchiveSource(archive_dir, mode="preserve"),
    dataset_name="imported_project_memory",
)
```

COGX has versioned readers and writers, one-way adapters, explicit import policies, archive and [network transport](#), [import summaries](#), and tested Cognition-to-Cognition graph-topology restoration. It proves this exchange layer is tractable without requiring Cognition or a graph database.

COGX 0.1 is an implementation foothold, not conformance to the six-part contract. Dataset export covers the graph, not complete original files, transcripts, vectors, or provenance ledgers. The schema has source identifiers and scopes, but those scopes do not enforce ACLs, and graph round trips lack complete typed-field fidelity and durable origin-to-local mappings; fact provenance is among the losses. Some identities are deterministic, but immutable revisions and supersession are absent.

Facts carry `valid_at` and `invalid_at`, but not lifecycle events, expiry, deletion coverage, or tombstones. Optional `permissions.json` is a superuser-gated Cognition restore containing credential material, not portable policy. The manifest rejects newer major versions and raw nodes retain unmapped properties, but there is no completeness, capability, required-extension, or object-loss declaration. Import summaries are receipt precursors.

This incompleteness makes the next work—durable identity, lineage, portable policy, lifecycle events, and fidelity reporting—concrete and testable. Existing [migration tests](#), [round-trip tests](#), and the [end-to-end restore](#) exercise the mechanics, not OMP conformance.

5. Standards above the common core

The core is a foundation, not the end of standardization. In the [RFC 6906](#) sense, a representation profile may add constraints and extensions without changing common semantics. Artifacts may declare several; the manifest must handle required, unknown, and conflicting profiles. A Markdown profile can define progressive disclosure, root context, metadata projection, and selective reads.

Runtime bindings can carry the same objects over archives, streams, sync, MCP, or AIDP-style federation, with capabilities, explicit read scopes and write destinations, and per-server results. Domain profiles may define vocabularies encoded in [JSON-LD](#) or evaluate [ODRL](#) for richer use policy. [me.md](#) and the AMS Card can add governance and disclosure. No layer may silently reinterpret the core.

6. Interoperability program and call to action

First publish a versioned schema and COGX, IBM, AIDP, and Markdown mappings, including unmappable fields. Public fixtures and a validator should exercise evidence and derivation, correction and consolidation, policy mapping, partial exports, conflicting revisions, required extensions, expiry, and deletion.

Two independent implementations must exchange them both ways without a shared engine or Cognee dependency. Published receipts must expose transformations, retries, missing evidence, denied policy mappings, and partial failures. A lifecycle demonstration covering correction, consolidation, invalidation, expiry, and deletion must include an old archive attempting to reintroduce a tombstoned revision.

Success means meaning and controls survive with limitations declared—not identical embeddings, prompts, ranking, graphs, or answers. COGX raw nodes remain an optional extension and are reported as opaque when uninterpretable. Fixtures and independent implementations should settle scope vocabulary, revision and event ordering, policy expressiveness, evidence references, and tombstone retention.

Requested decision: adopt the six-part portable object and exchange contract as OMP's common core; accept COGX as an implemented reference input and interoperability workbench; and develop Markdown loading as a profile and runtime APIs and federation as compatible bindings. We invite implementers to contribute mappings, fixtures, a second implementation, and adversarial transfer cases.

Review basis

Reviewed September 9, 2026 against the repository drafts, scoping note, research, and prototypes at [OMP commit a9b6ddf](#); the live [COGX documentation](#); and Cognee's implementation at [commit 352f02a](#). Proposed requirements state this contribution's design position. They are not claims of guarantees already provided across vendors.